

Implementing a Post-Quantum Secure Adaptor Signature Scheme in Blockchains

A lattice-based adaptor signature built on CRYSTALS-Dilithium, independently implemented in C and Rust and demonstrated in a post-quantum atomic swap.

Royce Steven Supervisor: Dr Zhipeng Wang

Quantum attacks are **not practical today**. Migration **cannot wait**.

< 500,000 physical qubits

estimated: minutes of runtime to recover a private key

secp256k1: the curve both chains sign with

Babbush et al. (2026)

| A blockchain never forgets, and fixing it needs **network agreement**, not one node.

| **NIST disallows it after 2035**

NIST IR 8547 ipd (2024) · 128-bit level, where secp256k1 sits

| **An estimate of cost, not a forecast of when** · no quantum computer this capable exists today.

Two strangers, two chains, no trusted middleman



Common adaptor constructions today are **elliptic-curve** based. **So why build a new one?**

Value-matched at spot · CoinGecko, 2026-08-21 11:57 UTC

Basic post-quantum signatures are standardised. Post-quantum adaptor signatures remain unevenly implemented.

Post-quantum adaptor signatures

support atomic swaps · **this project**

↑ implementations remain uneven (Buser et al., 2023)

Basic post-quantum signatures

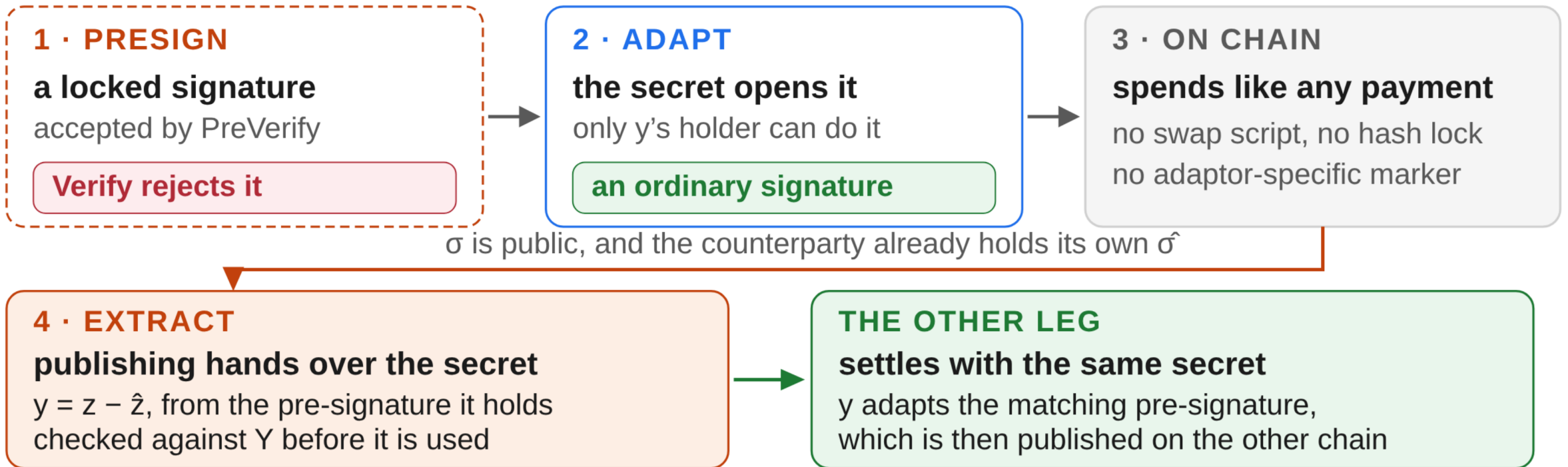
NIST FIPS 204 (2024) · Cloudflare ML-DSA authentication to origins, July 2026 (Valenta and Guthrie, 2026)

↑ migration under way, **and not only in blockchains** (Westerbaan, 2026)

Elliptic-curve signatures

what Bitcoin and Ethereum use today · broken by Shor

Publishing the completed signature **leaks the secret** to whoever holds the pre-signature.



Both legs settle, or neither does.

One substitution in the challenge hash. The lattice core is reused.

NEW: THE ADAPTOR LAYER

PreSign · PreVerify · Adapt · Extract
+ wire encoding, tests, and the swap application

SIMPLIFIED BASE SIGNATURE

KeyGen · Sign · Verify

REUSED UNCHANGED: CRYSTALS-Dilithium
NTT · polynomial arithmetic · SHAKE · sampling

zero upstream functions modified, shown by a two-branch diff

THE SUBSTITUTION

Sign $c = H(pk, w, M)$
PreSign $c = H(pk, w+Y, M)$

the statement, folded in ↑

H is instantiated in two steps:

$\tilde{c} = \text{SHAKE256}(pk, w[+Y], M)$

$c = \text{SampleInBall}(\tilde{c})$

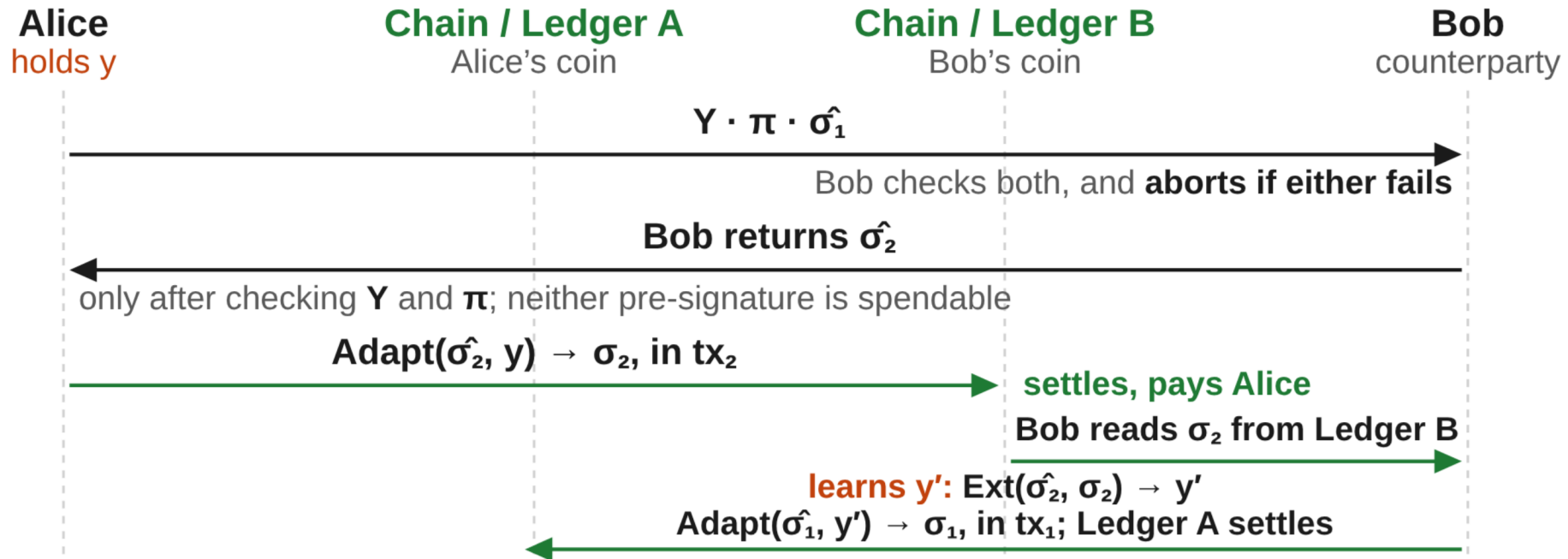
the digest $\tilde{c}$ travels on the wire

Built a second time, independently

C and Rust agree **byte for byte** on a pinned known-answer digest.

Two UTXO ledgers

The swap, step by step: settling one leg lets the other complete.



The signature published on **ledger B** is what hands Bob the secret for **ledger A**.

Post-quantum is the expensive step. The **adaptor layer** on top is not.

BASELINE ECDSA adaptor: the classical baseline

64 B

↓ step 1 · **already being taken:** quantum resistance, not adaptor functionality

STEP 1 LAS at Simplified Dilithium-II: the expensive step, and it is paid in **bytes**

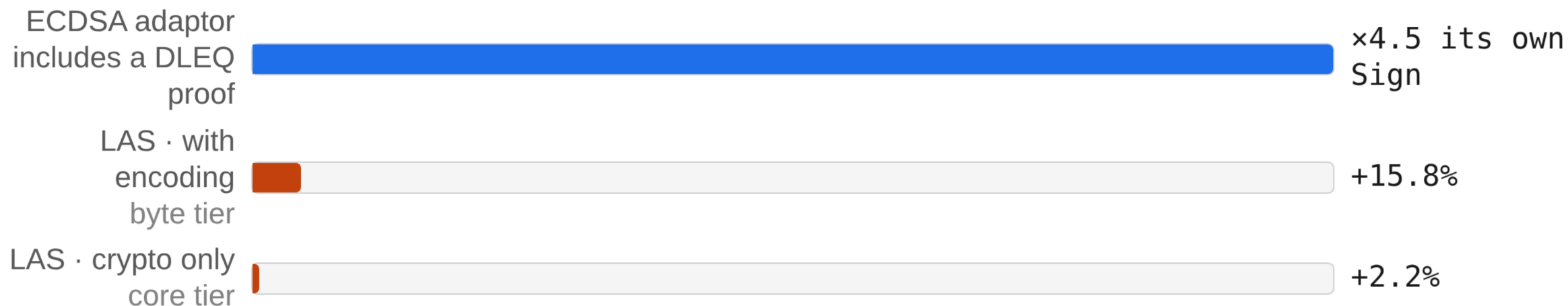
×72 vs ECDSA

↓ step 2 · **what this project measured:** against LAS's own post-quantum base

STEP 2 post-quantum ADAPTOR (LAS), at Simplified Dilithium-III: PreSign
+2.2% · PreVerify +5.1% · Adapt +7.4%

+8.2% max in sweep

Relative to its own base, the adaptor layer costs **less in LAS** than classically.



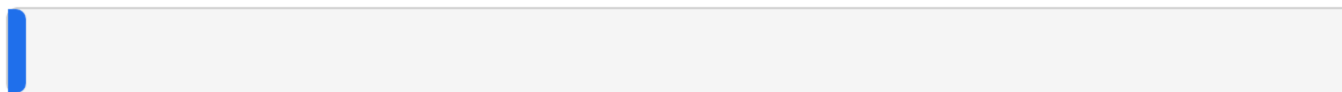
LAS: paired and interleaved against its own base · attempts **counted**, never inferred from timing. The classical figure is **derived** from a single mixed native-API tier.

The cost is not adaptor computation. It is **bytes on the wire**.

×72

larger signature · LAS vs ECDSA adaptor

ECDSA adaptor
compact signature



64 B

LAS
Simplified Dilithium-II



4640 B

Public key **×89** · **99.3%** of the signature is one object: the response **z**.

| **Fairness:** an ECDSA *adaptor* baseline, not plain ECDSA; matched to ≈128-bit by engineering, **not** by proof.

Same fields. One bigger slot. No new field.

Bitcoin	settled swap leg	Ethereum	the claim transaction
base: version · input · output · locktime	82 B	chain_id · nonce · fees · to = escrow · value	envelope
witness: σ + public key in ≤ 520 B elements · tapscript · control block	11,289 B	signature: still ECDSA over secp256k1, consensus checks this	65 B
$Y \cdot \sigma \cdot \pi \cdot y$	never on chain	data: σ + verifier inputs	50,148 B
MEASURED mined, client-reported · 2,905 vB · regtest, experimental rule		$Y \cdot \sigma \cdot \pi \cdot y$	never on chain
		MEASURED a real client's receipt · 97.8% of the per-transaction cap	

Bitcoin is **more restricted**: native LAS verification needs a **consensus rule**. Ethereum is **more flexible**: verification can be **deployed as a contract**, at a gas cost.

Size is not enough. Stock Bitcoin does not verify LAS, so I added an experimental rule.

one spend · 2,917 vB · no elliptic-curve signature · two clients, same release

Carriage on a **stock** node: REFUSED by relay policy · VALID by consensus · MINED

patched Core v31.1 0xbb verifies the LAS signature

ACCEPTED · **MINED** the valid spend

REJECTED ×9 every negative control

stock Core v31.1 0xbb is still OP_SUCCESS

ACCEPTED the valid spend · same block

ACCEPTED ×9 every negative control

The difference between the two nodes is the new rule, nothing else.

| A patched node is **not** Bitcoin · the rule's security is **not** analysed.

Verification fits in **one transaction**, with almost no margin.

Does full verification fit in one transaction?

D3 · whole
claim

97.8%

track = EIP-7825 per-transaction cap

Could these wait as future work? Tested instead

statement truncation: fails at Extract, every depth tested

the tested succinct-PQ route: loses to the deployed prover at this relation size

One signature instance · local EVM, pinned **Osaka** revision.

The original LAS construction uses a **simplified base** (Esgin et al., 2020). Functionally, the unchanged ML-DSA-65 verifier accepts adapted signatures.

PreSign: new **PreVerify: new** **Verify: unchanged**

No, functionally: on the full FIPS 204 ML-DSA-65 path the stock verifier accepts **200/200** adapted signatures.

	simplified DIII	ML-DSA-65
signature	6736 B	3309 B
statement Y	4416 B	4416 B

Y is unchanged: the **dominant size target** at the signature layer.

| ML-DSA build is **functional only** · its security is **not** analysed.

Deployment is constrained by verification cost and platform integration, not by adaptor-layer overhead.

Bitcoin

Native LAS verification needs a **consensus rule**, not engineering.

374 μ s

per input, once the rule exists

the rule is **unanalysed**

Ethereum

Fits **today** in one EVM transaction, with almost no margin.

97.8%

of the per-transaction cap, whole claim

one signature instance

Protocol design

Budget the **proof** first.

98.6%

of end-to-end time is the proof

optimise π and Y , not the adaptor